
PROJETO 1 DA DISCIPLINA DE INF2608 – FUNDAMENTOS DE COMPUTAÇÃO GRÁFICA

Análise Técnico-Científica de uma Implementação de Ray Tracing

Technical-Scientific Analysis of a Ray Tracing Implementation

Yang Ricardo Barcellos Miranda [ymiranda@inf.puc-rio.br]

Resumo. Este artigo apresenta uma análise técnico-científica de uma implementação de ray tracing desenvolvida no Projeto 1 da disciplina de INF2608. O objetivo é verificar, a partir do código efetivamente executável, quais componentes geométricos, fotométricos e amostrais foram materializados, como eles se relacionam com os materiais da disciplina e em que medida a implementação se aproxima das referências selecionadas de PBRT 4e. A análise evidencia um núcleo sólido de câmera pinhole, raio paramétrico, interseções analíticas, sombreado local de Phong e sombras diretas, além de extensões como supersampling, instanciamento afim, luz de área, reflexão com Fresnel-Schlick, refração com Lei de Snell, reflexão interna total, Beer-Lambert e transmitância acumulada em raios de sombra. Para triângulos e aceleração, o artigo distingue cuidadosamente entre o conteúdo teórico e o que está implementado: há suporte a **Triangle**, **TriangleMesh** e uma BVH estática local para malhas, mas não há um acelerador global de cena. O trabalho também discute a infraestrutura experimental do projeto, com CLI padronizada, persistência de artefatos de render e diagramas que reforçam a rastreabilidade entre formulação teórica, estrutura algorítmica e evidência visual.

Abstract. This paper presents a technical-scientific analysis of a ray tracing implementation developed for the first INF2608 course project. Its main goal is to verify, from the executable code itself, which geometric, photometric, and sampling components were effectively implemented, how they relate to the course material, and to what extent the result aligns with selected PBRT 4e references. The analysis shows a solid core built around a pinhole camera, parametric rays, analytic intersections, local Phong shading, and direct shadows, as well as extensions such as supersampling, affine instancing, area lights, Fresnel-Schlick reflection, dielectric refraction with Snell's law, total internal reflection, Beer-Lambert attenuation, and accumulated transmittance for shadow rays. For triangles and acceleration, the paper carefully distinguishes between theoretical coverage and implemented functionality: the codebase supports **Triangle**, **TriangleMesh**, and a static local BVH for meshes, but not a global scene accelerator. The article also discusses the project's experimental infrastructure, including a standardized CLI, persistent render artifacts, and diagrams that strengthen the traceability between theoretical formulation, algorithmic structure, and visual evidence.

Palavras-chave: traçado de raios, computação gráfica, PBRT, BVH, amostragem, renderização fisicamente inspirada

Keywords: ray tracing, computer graphics, PBRT, BVH, sampling, physically inspired rendering

Entrega: 10/05/2026

1 Introdução

Este artigo apresenta uma análise técnico-científica da implementação de ray tracing disponível no repositório do Projeto 1 [Miranda, 2026]. O objeto principal da análise é o código em `src/ray_tracing_2/`, confrontado com os tópicos dos slides da disciplina, de autoria de Waldemar Celes (W. Celes, celes@inf.puc-rio.br) [Celes, 2026b,c,a], e com referências selecionadas de *Physically Based Rendering* (PBRT 4e), em especial [Pharr *et al.*, 2023, Seções 1.2, 3.5, 3.7, 5.2, 7.3, 8.1, 8.5, 9.3, 9.5, 11.2 e 12.4].

O objetivo não é recontar genericamente o que um ray tracer deveria fazer, mas verificar o que esta implementação entrega de forma observável, quais decisões algorítmicas sustentam cada efeito visual e em que pontos o projeto simplifica ou parcializa o conteúdo teórico. A leitura adotada separa três camadas: a formulação

geométrica e física discutida em aula, a tradução algorítmica efetivamente materializada no repositório e as lacunas entre a teoria de referência e o escopo implementado.

Ao longo do texto, os slides da disciplina, o PBRT 4e e o repositório aparecem como referências formais. Já a rastreabilidade interna do código é apresentada por caminho relativo, símbolo e linha, de modo a permitir inspeção local das decisões relevantes sem confundir documentação bibliográfica com evidência de implementação.

A organização da análise segue quatro frentes complementares:

1. reconstituir o núcleo geométrico e fotométrico do traçador, com raio paramétrico, interseções, câmera pinhole, Phong e sombras;
2. examinar as extensões ópticas e amostrais, inclu-

- indo supersampling, instanciação, luz de área, reflexão, refração e transmitância;
- 3. avaliar a geometria triangular e a aceleração local por BVH, distinguindo implementação real de cobertura apenas teórica;
- 4. consolidar experimentos breves, limitações atuais, diagramas de apoio e uma avaliação final do que a implementação entrega.

Como fonte externa, foi usado apenas o conteúdo de *Physically Based Rendering* (PBRT 4e), com ênfase em câmera projetiva, teoria de amostragem, reflexão e transmissão especular, transmitância, luzes de área, caixas envoltantes e BVHs. A edição consultada foi a quarta edição online dessa obra [Pharr *et al.*, 2023].

2 Núcleo Geométrico e Fotométrico do Traçador

2.1 Conceitos Estruturantes do Núcleo de Referência

O material introdutório de traçado de raios [Celes, 2026b] organiza-se em torno de um conjunto pequeno de conceitos que estrutura todo o traçador de raios.

1. **Raio paramétrico e visibilidade (pp. 7-9).** A formulação $\mathbf{r}(t) = \mathbf{o} + t\hat{\mathbf{d}}$ reduz a linha de visada a uma reta orientada; t funciona como distância assinada ou paramétrica ao longo do feixe, e a visibilidade do ponto observado é determinada pelo menor $t > 0$ compatível com uma superfície.
2. **Interseções analíticas e controle numérico (pp. 10-18).** O plano é descrito por $(\mathbf{x} - \mathbf{p}_0) \cdot \hat{\mathbf{n}} = 0$, enquanto a esfera leva à quadrática $at^2 + bt + c = 0$, cujo discriminante Δ classifica ausência, tangência ou dupla interseção; do ponto de vista computacional, essa geometria analítica só é robusta quando acompanhada por um critério de tolerância ε para rejeitar paralelismos numéricos e auto-interseções.
3. **Câmera pinhole, filme e mudança de base (pp. 19-39).** A câmera é construída por uma base ortogonal $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \hat{\mathbf{w}})$ e pela geometria do filme, com $\Delta v = f \tan(\theta/2)$ e $\Delta u = \Delta v (w/h)$; em síntese, esse bloco combina geometria projetiva com mudança de base entre espaço da câmera e espaço global.
4. **Iluminação local e fotometria simplificada (pp. 40-43).** O modelo local de Phong combina a componente difusa lambertiana $\max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$ com a componente especular $\max(0, \hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^s$; fisicamente, trata-se de um modelo local e fenomenológico, útil para capturar dependência angular sem resolver transporte global de luz.
5. **Organização algorítmica da cena, sombras e luz ambiente (pp. 44-55).** O pipeline de *closest hit* seleciona a primeira interseção visível, o raio de sombra transforma a iluminação direta em um teste de visibilidade até a fonte, e o termo ambiente $m_{amb} l_{amb}$ fecha o modelo como aproximação empírica da iluminação indireta ausente.

Esses conceitos explicam por que as subseções seguintes podem ser lidas como desdobramentos detalhados do mesmo núcleo conceitual.

2.2 Raio paramétrico, visibilidade e registro de interseção

O ponto de partida do Slide 4 é a modelagem do raio por

$$\mathbf{r}(t) = \mathbf{o} + t\hat{\mathbf{d}}, \quad t \geq 0,$$

em que $\mathbf{o}$ é a origem, $\hat{\mathbf{d}}$ é a direção normalizada e t parametriza a posição ao longo do feixe. Em ótica geométrica, essa formulação abstrai a propagação retilínea da luz em meios homogêneos; em computação gráfica, ela converte o problema de visibilidade em um problema de encontrar o menor t positivo compatível com uma superfície. Nos slides, isso aparece como o elo entre câmera, interseção e sombreamento local, particularmente na dedução do raio paramétrico e na interpretação física de t como distância assinada ao longo da trajetória [Celes, 2026b, pp. 7–9].

O repositório segue exatamente essa organização. O raio é carregado por `Ray`, enquanto `Hit` armazena a melhor interseção conhecida até o momento e também codifica `front_face` e `backfacing`. Essa distinção, que no Slide 4 serve para consistência geométrica da normal, torna-se decisiva no Slide 5 para distinguir entrada e saída de meios refrativos. A rotina `Scene.compute_intersection()` preserva o padrão clássico de *closest hit*: percorre os objetos, atualiza um único registro acumulador e retorna apenas a interseção frontal mais próxima.

Rastreabilidade no código. `src/ray_tracing_2/hit.py` (`Hit`, l. 11; `set_face_normal()`, l. 25), `src/ray_tracing_2/scene.py` (`compute_intersection()`, l. 24; `trace_ray()`, l. 102).

Ver também [Pharr *et al.*, 2023, Seções 1.2 e 3.5].

2.3 Interseções básicas: plano, esfera e controle numérico de ε

Nos slides, o plano é descrito pela equação implícita

$$(\mathbf{x} - \mathbf{p}_0) \cdot \hat{\mathbf{n}} = 0,$$

e a substituição de $\mathbf{r}(t)$ produz a solução linear

$$t = \frac{(\mathbf{p}_0 - \mathbf{o}) \cdot \hat{\mathbf{n}}}{\hat{\mathbf{d}} \cdot \hat{\mathbf{n}}},$$

exatamente como discutido nas páginas de interseção raio-plano [Celes, 2026b, pp. 10–13]. A esfera, por sua vez, impõe

$$\|\mathbf{x} - \mathbf{c}\|^2 - r^2 = 0,$$

que, após substituição do raio, gera uma quadrática $at^2 + bt + c = 0$ cujo discriminante $\Delta = b^2 - 4ac$ decide ausência, tangência ou dupla interseção [Celes, 2026b, pp. 14–18]. Em ambos os casos, o aspecto fisicamente relevante é o mesmo: o que interessa para a câmera é

a menor raiz positiva. O aspecto numericamente relevante também é comum: deve-se rejeitar soluções com t demasiado pequeno para evitar reinterseção artificial da própria superfície.

O código espelha esse raciocínio. `Plane.intersect()` resolve o plano por produto escalar; `Sphere.intersect()` resolve a quadrática e guarda a menor raiz positiva; `Scene.offset_point()` empurra a origem dos raios secundários ao longo da normal geométrica com `ray_epsilon = 0.001`, evitando *shadow acne* em sombra, reflexão e refração. O Slide 4 apresenta essa ideia como tolerância numérica; o repositório a aplica de forma centralizada, o que é arquiteturalmente melhor do que espalhar pequenos limiares por todos os materiais.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`Sphere`, l. 17; `Sphere.intersect()`, l. 23; `Plane`, l. 54; `Plane.intersect()`, l. 60), `src/ray_tracing_2/scene.py` (`offset_point()`, l. 33).

Ver também [Pharr *et al.*, 2023, Seção 1.2].

2.4 Câmera pinhole, filme e espaços de coordenadas

O Slide 4 reconstrói o modelo pinhole por uma base ortonormal de câmera, usando `eye`, `center` e `up` para montar os eixos locais (pp. 19–23), e em seguida projeta amostras do pixel sobre o plano de imagem (pp. 24–29), antes de explicitar a mudança entre espaço de câmera e espaço global (pp. 31–39) [Celes, 2026b]. A formulação algébrica é a de uma mudança de base: do espaço local da câmera para o espaço global da cena. Em uma formulação clássica, isso aparece como matriz de visualização, sua inversa e projeção perspectiva parametrizada por campo de visão, aspecto e distância ao plano de imagem.

No código, a ideia é a mesma, mas a implementação é condensada: `Camera.__init__()` usa `glm.lookAt()` para obter a transformação de visualização e armazena sua inversa em `inv_view`. A geração do raio em `generate_ray()` monta o ponto do pixel em espaço de câmera a partir de coordenadas normalizadas e da relação trigonométrica

$$\Delta v = f \tan\left(\frac{\theta}{2}\right), \quad \Delta u = \Delta v \frac{w}{h},$$

que é precisamente a síntese entre geometria projetiva e semelhança de triângulos apresentada nos slides (pp. 24–29) e depois transforma esse ponto para o espaço global (pp. 31–39) [Celes, 2026b]. O comentário recém-ajustado em `camera.py` deixa explícita uma nuance importante: `focal_distance` aqui é distância geométrica ao plano de projeção no modelo pinhole, não distância focal de lente fina nem parâmetro de profundidade de campo. Esse detalhe é essencial para não projetar sobre o código uma física que ele não implementa.

Essa distinção também define uma limitação importante do projeto: todos os raios primários partem de um

único centro ótico, sem abertura finita, círculo de confusão ou profundidade de campo. Portanto, o antialiasing implementado depois no Slide 5 altera a estratégia de amostragem do pixel, mas não muda o modelo óptico de formação de imagem.

Em `film.py`, `get_sample()` e `render()` deixam claro que a imagem final é construída como média de amostras por pixel. Mesmo quando o caso básico usa apenas uma amostra central, a interface já está preparada para supersampling estocástico.

Rastreabilidade no código. `src/ray_tracing_2/camera.py` (`Camera`, l. 6; `__init__()`, l. 7; `generate_ray()`, l. 23), `src/ray_tracing_2/film.py` (`Film`, l. 20; `get_sample()`, l. 54; `render()`, l. 108), `src/ray_tracing_2/main.py` (`render()`, l. 25).

Ver também [Pharr *et al.*, 2023, Seção 5.2].

2.5 Phong, luz pontual, termo ambiente e sombra dura

O Slide 4 fecha o núcleo do renderizador com o modelo local de Phong, isto é, uma soma de termo ambiente, componente difusa lambertiana e componente especular dependente do vetor refletido [Celes, 2026b, pp. 40–43, 54–55]. Em termos físicos, esse modelo não é um BRDF rigoroso, mas é uma aproximação útil: o termo difuso usa o fator $\max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$; o termo especular usa um pico angular em função de $\max(0, \hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^s$; a sombra dura decorre de um teste binário de visibilidade entre o ponto sombreado e a fonte [Celes, 2026b, pp. 51–53]. Em particular, as páginas 40–43 articulam a síntese física mínima do projeto: incidência luminosa, orientação local de normal e dependência angular da resposta da superfície.

`PhongMaterial.direct_lighting()` implementa exatamente esse cálculo. Para cada luz, o código obtém radiância incidente e direção via `sample_radiance()`, soma o termo difuso e depois o termo especular. `Scene.transmittance()` faz o papel do raio de sombra, mas já em uma forma mais geral: para meios opacos, ele reduz-se ao teste binário do Slide 4; para meios transparentes, ele vira um produto acumulado de transmittâncias, o que antecipa o Slide 5.

Há, contudo, uma nuance importante entre a física dos slides e a convenção da implementação: os slides apresentam a lei usual de decaimento geométrico de uma fonte pontual, proporcional a $1/r^2$, mas `PointLight.radiance()` não aplica esse fator. O comentário do código explicita a razão: o projeto preserva a convenção do enunciado em que `power` é tratado como radiância constante da fonte. Isso não é um erro, mas uma simplificação de modelagem que precisa ser declarada no relatório para evitar falsa equivalência com uma formulação radiométrica completa.

Rastreabilidade no código. `src/ray_tracing_2/light.py` (`PointLight`, l. 42; `PointLight.radiance()`, l. 46), `src/ray_tracing_2/material.py` (`PhongMaterial`, l. 37; `direct_lighting()`, l. 45; `eval()`, l. 72), `src/ray_tracing_2/scene.py` (`transmittance()`, l. 48), `src/ray_tracing_2/main.py` (`Sphere`, l. 54; `Plane`, l. 55; `PointLight`, l. 59).

Ver também [Pharr *et al.*, 2023, Seção 1.2].

A síntese consolidada de aderência entre o núcleo geométrico-fotométrico de referência e a implementação atual foi deslocada para a Seção 9, de modo a preservar a continuidade argumentativa desta parte analítica.

3 Extensões Ópticas, Amostrais e de Transformação

3.1 Antialiasing como integração estocástica sobre o pixel

O Slide 5 reinterpreta o pixel não como uma posição única, mas como um domínio de amostragem no qual múltiplos raios podem ser gerados [Celes, 2026c, pp. 4–8]. A leitura correta em termos numéricos é de integração de Monte Carlo: a cor do pixel passa a ser estimada pela média de amostras da radiancia incidente,

$$\hat{L} = \frac{1}{N} \sum_{k=1}^N L(x_k),$$

convertendo aliasing estruturado em ruído de alta frequência. Os slides enfatizam o caso aleatório uniforme e a fórmula de jitter subpixel nas páginas 4-9.

O repositório implementa duas versões dessa ideia em `Film.get_samples_for_pixel()`: `jittered` e `stratified`. A primeira preserva a interpretação direta dos slides: amostras aleatórias independentes no interior do pixel. A segunda é uma extensão consistente com [Pharr *et al.*, 2023, Seção 8.5]: o pixel é subdividido em subcélulas e cada subcélula recebe uma amostra com jitter interno, o que reduz variância sem abandonar o caráter estocástico do estimador. Após a refatoração recente, a definição concreta desses padrões 2D foi extraída para `src/ray_tracing_2/sampling.py`, de modo que `film.py` passou a atuar como camada de interpretação geométrica do domínio do pixel, e não mais como o único lugar onde os padrões são definidos. Isso fortalece a correspondência entre a teoria de amostragem do Slide 5 e a arquitetura do código: há um mesmo quadrado unitário amostral, mas cada domínio o reinterpreta de forma física distinta.

Rastreabilidade no código. `src/ray_tracing_2/sampling.py` (`uniform_samples_2d()`, l. 15; `stratified_grid_samples_2d()`, l. 42), `src/ray_tracing_2/film.py` (`get_samples_for_pixel()`, l. 61; `render()`, l. 112).

Ver também [Pharr *et al.*, 2023, Seções 8.1 e 8.5].

3.2 Instanciação, coordenadas homogêneas e transformação correta de normais

O Slide 5 também formaliza a ideia de instância geométrica: em vez de reescrever a interseção de um elipsoide, por exemplo, transforma-se o raio do espaço global para o espaço local de uma primitiva canônica [Celes, 2026c, pp. 9–13]. Em álgebra linear, isso significa representar translação, rotação e escala em coordenadas homogêneas e aplicar a inversa da transformação ao raio inci-

dente. A síntese mais importante desse bloco aparece na dedução da transformação de normais por

$$\mathbf{n}' = (M^{-1})^T \mathbf{n},$$

preservando a ortogonalidade com o plano tangente após transformações afins gerais.

`Instance.intersect()` implementa exatamente esse fluxo. O raio é mapeado com `m_inv`, a interseção é resolvida no espaço local da forma base e, se houver hit, a posição volta ao mundo pela matriz direta. A normal, porém, usa `m_inv_t`, isto é, a inversa transposta. Essa é a consequência algébrica correta da preservação da ortogonalidade entre normal e plano tangente. O relatório precisa explicitar esse ponto porque ele é um dos poucos trechos em que a implementação realmente depende de um fato sutil de álgebra linear, e não apenas de manipulação vetorial elementar.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`Instance`, l. 253; `Instance.intersect()`, l. 262), `src/ray_tracing_2/main_box.py` (`render()`, l. 63), `src/ray_tracing_2/main_triangles.py` (`render()`, l. 100).

Ver também [Pharr *et al.*, 2023, Seção 3.5].

3.3 Luz de área, integração sobre o emissor e penumbra

Nos slides, a luz de área substitui a fonte pontual idealizada por um emissor com extensão geométrica finita, o que exige integrar a contribuição luminosa sobre uma superfície [Celes, 2026c, pp. 14–23]. Essa mudança é fisicamente relevante: penumbra surge porque diferentes subregiões da fonte são visíveis ou ocluídas de maneira diferente a partir do ponto sombreado. As páginas 14–17 introduzem o emissor extenso; as páginas 18–23 traduzem isso em soma de amostras e variabilidade espacial da visibilidade.

`AreaLight.sample_radiance()` aproxima essa integral por soma discreta sobre uma grade `samples_u × samples_v`, com jitter em cada célula. A construção é coerente com os slides e com [Pharr *et al.*, 2023, Seção 12.4]: a energia é distribuída entre as amostras e sofre decaimento geométrico explícito com $1/r^2$, diferentemente da `PointLight`. Depois da extração de `src/ray_tracing_2/sampling.py`, os padrões `uniform`, `regular` e `stratified` passaram a ser definidos em um único módulo interno e a `AreaLight` apenas os converte do quadrado unitário para a superfície emissiva por meio da parametrização afim $p + u e_u + v e_v$. Em outras palavras, a implementação preserva a convenção simplificada para luz pontual, mas usa uma aproximação radiometricamente mais próxima do caso físico quando a fonte possui área.

Essa dualidade precisa ser registrada: o projeto não usa um único modelo coerente para todas as luzes. Ele usa, deliberadamente, a convenção do enunciado para `PointLight` e uma aproximação mais geométrica para `AreaLight`.

Rastreabilidade no código. `src/ray_tracing_2/sampling.py` (`uniform_samples_2d()`, l. 1).

```
15; regular_grid_samples_2d(), l. 26;
stratified_grid_samples_2d(), l. 42),
src/ray_tracing_2/light.py (AreaLight,
l. 80; _iter_sample_uvs(), l. 114;
sample_radiance(), l. 133; radiance(), l. 173),
src/ray_tracing_2/main_area_light.py (render(),
l. 31; criação da AreaLight, l. 59-66).
```

Ver também [Pharr *et al.*, 2023, Seção 12.4].

3.4 Caixa, AABB e método de slabs

O Slide 5 introduz a caixa alinhada aos eixos tanto como primitiva quanto como ferramenta de raciocínio geométrico [Celes, 2026c, pp. 24–25]. A essência do método de slabs é decompor a interseção do raio com a caixa em três intervalos unidimensionais, um por eixo, e combinar os tempos de entrada e saída por máximos e mínimos. Isso é uma interseção de intervalos, não um teste de plano isolado.

O repositório usa essa ideia em dois níveis. Primeiro, `Box.intersect()` implementa a caixa como primitiva visível da cena. Segundo, `AABB.intersects()` em `triangle_bvh.py` reutiliza o mesmo princípio como teste de poda para a BVH de triângulos. Esse reuso é conceitualmente importante: o Slide 5 ensina uma primitiva; o Slide 6 reaproveita exatamente a mesma álgebra como volume de contenção.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`Box`, l. 74; `Box.intersect()`, l. 88), `src/ray_tracing_2/triangle_bvh.py` (`AABB`, l. 47; `intersects()`, l. 60).

Ver também [Pharr *et al.*, 2023, Seção 3.7].

3.5 De traçador local a traçador recursivo

O salto conceitual mais importante do Slide 5 é que o pipeline do Slide 4 não é descartado; ele é recursivamente reusado [Celes, 2026c, pp. 26–35]. Em termos da equação de transporte, isso equivale a acrescentar alguns caminhos óticos privilegiados ao estimador: reflexão perfeita, transmissão perfeita e luzes explícitas. Em termos algorítmicos, significa que certos materiais deixam de devolver apenas uma cor local e passam a disparar novos raios. As páginas 26–28 cobrem a reflexão, as páginas 29–34 cobrem a transmissão, e a página 35 consolida a generalização do shadow ray em transmissão acumulada.

O repositório codifica essa extensão com clareza. `Scene.trace_ray()` continua mínimo: encontra o `Hit` e delega ao material. A recursão é controlada por `Scene.can_spawn_ray()`, que impõe profundidade máxima finita. Isso é coerente tanto numericamente quanto fisicamente: cada salto adicional é ponderado por refletância ou transmitância menores que 1, de modo que a contribuição tende a decair. A limitação de profundidade, portanto, é ao mesmo tempo um controle de custo e uma aproximação de truncamento de série.

Rastreabilidade no código. `src/ray_tracing_2/scene.py` (`can_spawn_ray()`, l. 41; `trace_ray()`, l. 102).

Ver também [Pharr *et al.*, 2023, Seção 1.2].

3.6 Reflexão especular com Fresnel-Schlick

O Slide 5 introduz a reflexão metálica via aproximação de Schlick, isto é,

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5,$$

como aproximação de baixo custo das equações de Fresnel para interfaces suaves [Celes, 2026c, pp. 26–28]. O papel físico de R_0 é resumir a refletância em incidência normal; o papel geométrico de θ é medir a inclinação relativa entre direção de visão e normal. As páginas 26–27 são as mais importantes aqui: nelas os slides ligam a expressão de Schlick ao rateio entre resposta local e raio refletido recursivo.

`ReflectiveMaterial.eval()` implementa exatamente esse rateio: a componente local de Phong é ponderada por $(1 - R)$ e a contribuição do raio refletido é ponderada por R . É importante, contudo, qualificar a frase “conservação de energia” que aparece nos slides-resumo. No repositório, essa mistura é fisicamente plausível, mas não é um BRDF estrito nem um balanço energético formal no sentido de PBRT. O termo local de Phong permanece empírico, e o uso de Schlick aqui deve ser entendido como melhora angular do modelo, não como substituição integral por um material fisicamente baseado.

Há ainda uma nuance angular importante. O cosseno usado na aproximação de Schlick vem da inclinação entre a direção de observação e a normal local, isto é, de um termo do tipo `dot(v, n)`, e não de um “ângulo crítico” pré-computado da interface. Essa escolha é consistente com a forma clássica de Schlick para refletância direcional, mas reforça o caráter aproximado do material: a implementação melhora a dependência angular da reflexão sem migrar para um modelo microfacet completo.

Rastreabilidade no código. `src/ray_tracing_2/material.py` (`ReflectiveMaterial`, l. 84; `eval()`, l. 101).

Ver também [Pharr *et al.*, 2023, Seção 9.3].

3.7 Refração dielétrica, Lei de Snell, reflexão interna total e Beer-Lambert

O bloco de refração dos slides [Celes, 2026c, pp. 29–36] é o trecho de física mais rico do projeto. A interface entre meios com índices de refração diferentes obedece a Snell,

$$\eta_i \sin \theta_i = \eta_t \sin \theta_t,$$

enquanto a fração refletida continua sendo modulada por Fresnel. As páginas 29–32 concentram a dedução geométrica da refração a partir da decomposição em componentes normal e tangencial; a página 33 introduz Beer-Lambert; a página 34 sintetiza o algoritmo completo do dielétrico. Além disso, quando o radicando da solução vetorial de transmissão fica negativo, ocorre reflexão interna total. Finalmente, se o raio efetivamente penetra no material, a intensidade transmitida

deve ser atenuada pela espessura percorrida, segundo Beer-Lambert.

`TransparentMaterial.eval()` implementa essa sequência de decisões. Primeiro calcula R_0 e R por Schlick; depois escolhe a razão η_i/η_t a partir de `front_face` e `backfacing`; em seguida chama `glm.refract()` e interpreta um vetor nulo como TIR; por fim, aplica a atenuação por canal

$$I(\lambda, s) = I_0(\lambda) a(\lambda)^s.$$

O uso de `front_face` e `backfacing`, calculado em `Hit.set_face_normal()`, é crucial: ele traduz a orientação geométrica da interface em semântica ótica de entrada e saída do meio. Isso é um caso exemplar de acoplamento correto entre álgebra linear local da superfície e física de propagação.

No caso da iluminação direta através de transparentes, `TransparentMaterial.shadow_transmittance()` aplica Beer-Lambert apenas quando o raio de sombra deixa o meio, isto é, no caso `backfacing = True`. Essa escolha é coerente com a hipótese de meio homogêneo adotada pelo projeto: o comprimento óptico relevante já foi acumulado em `hit.t` no segmento interno do material, de modo que atenuar também na entrada equivaleria a dupla contagem. Trata-se, ainda assim, de uma simplificação relevante: o modelo opera por canal RGB, sem dispersão espectral contínua e sem heterogeneidade volumétrica interna. Em termos de física computacional, isso equivale a resolver um meio homogêneo com profundidade óptica escalar por canal, e não um meio espectral completo.

Rastreabilidade no código. `src/ray_tracing_2/hit.py` (`set_face_normal()`, l. 25), `src/ray_tracing_2/material.py` (`TransparentMaterial`, l. 137; `shadow_transmittance()`, l. 167; `eval()`, l. 186).

Ver também [Pharr *et al.*, 2023, Seções 9.3, 9.5 e 11.2].

3.8 Da sombra binária à transmitância acumulada

O Slide 5 culmina em uma generalização conceitualmente elegante: o antigo raio de sombra do Slide 4 deixa de responder apenas “visível” ou “ocluído” e passa a acumular atenuações ao atravessar materiais transparentes [Celes, 2026c, pp. 33–35]. Em termos físicos, esse é um primeiro passo em direção à ideia mais geral de transmitância ao longo de um segmento, isto é, à acumulação multiplicativa de fatores do tipo

$$T = \prod_k a_k^{s_k}.$$

Em termos computacionais, é uma iteração sobre hits sucessivos até a fonte.

`Scene.transmittance()` implementa precisamente essa lógica. O segmento entre ponto sombreado e luz é percorrido em vários passos; objetos opacos zeram a energia; materiais transparentes delegam a `shadow_transmittance()`, que no caso dielétrico

aplica Beer-Lambert apenas ao sair do meio. O algoritmo ainda adiciona dois cuidados numéricos: limiar inferior para throughput muito pequeno e limite `max_steps` para evitar laços indevidos em cenas mal condicionadas.

Esse método é particularmente importante porque mostra a coerência arquitetural do projeto: o mesmo mecanismo de interseção da cena é reutilizado tanto para visibilidade direta quanto para óptica recursiva.

Rastreabilidade no código. `src/ray_tracing_2/scene.py` (`transmittance()`, l. 48), `src/ray_tracing_2/material.py` (`Material.shadow_transmittance()`, l. 30; `TransparentMaterial.shadow_transmittance()`, l. 167), `src/ray_tracing_2/light.py` (`PointLight.radiance()`, l. 46; `AreaLight.sample_radiance()`, l. 92).

Ver também [Pharr *et al.*, 2023, Seção 11.2].

A síntese consolidada de aderência entre as extensões ópticas e amostrais de referência e a implementação atual foi reunida na Seção 9, junto ao fechamento do artigo.

4 Triângulos e Estruturas de Aceleração

4.1 Triângulo, área orientada e coordenadas baricêntricas

O Slide 6 começa corretamente pelo triângulo como primitiva elementar de malhas [Celes, 2026a, pp. 3–4]. Os vetores de aresta $\mathbf{e}_1 = \mathbf{b} - \mathbf{a}$ e $\mathbf{e}_2 = \mathbf{c} - \mathbf{a}$ definem uma área orientada via produto vetorial,

$$A = \frac{\|\mathbf{e}_1 \times \mathbf{e}_2\|}{2},$$

e qualquer ponto do triângulo pode ser expresso por coordenadas baricêntricas,

$$\mathbf{p} = \mathbf{a} + u\mathbf{e}_1 + v\mathbf{e}_2, \quad u \geq 0, v \geq 0, u + v \leq 1.$$

A importância disso vai além da geometria elementar: coordenadas baricêntricas são a ponte entre interior do triângulo, teste de pertencimento e parametrização afim da superfície.

No código, `Triangle` pré-computa $\mathbf{e}_1$, $\mathbf{e}_2$ e a normal geométrica orientada a partir de `cross(e1, e2)`. A normal degenerada é rejeitada com exceção explícita, o que evita triângulos colineares ou nulos no carregamento da malha.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`Triangle`, l. 144).

4.2 Interseção raio-triângulo por Möller-Trumbore

O coração algébrico do Slide 6 é a dedução do teste de Möller-Trumbore a partir do sistema linear que iguala ponto do raio e combinação baricêntrica dos vértices [Celes, 2026a, pp. 5–12]. Em notação compacta, os slides resolvem algo do tipo

$$\mathbf{o} + t\hat{\mathbf{d}} = \mathbf{a} + u\mathbf{e}_1 + v\mathbf{e}_2,$$

via determinantes e produtos mistos, e a solução só é válida se $u \geq 0$, $v \geq 0$, $u + v \leq 1$ e $t > 0$.

`Triangle.intersect()` implementa exatamente essa forma operacional: `pvec`, `det`, `inv_det`, `u`, `v`, `qvec` e `t_candidate`. Em termos geométricos, o algoritmo evita construir explicitamente o plano do triângulo; em termos computacionais, ele é compacto e numericamente robusto para o porte do projeto. O teste `abs(det) < eps` trata o caso de quase paralelismo entre raio e plano do triângulo.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`Triangle.intersect()`, l. 158).

4.3 Malha triangulada e carregamento de geometria

Os slides seguem do triângulo isolado para a malha: tabela de vértices, tabela de incidência e necessidade de estruturas de aceleração [Celes, 2026a, pp. 13–14]. O repositório segue essa linha em `TriangleMesh`, que pode ser construído diretamente de listas de vértices e faces e ainda ser instanciado por `Instance`, preservando o mesmo pipeline de interseção e material das primitivas analíticas.

Essa homogeneidade é importante: a malha não cria um segundo sistema de renderização. Ela é apenas outra fonte de primitivas compatíveis com `Scene.compute_intersection()`. A aceleração, quando presente, muda custo computacional, não semântica geométrica do hit.

Rastreabilidade no código. `src/ray_tracing_2/shape.py` (`TriangleMesh`, l. 188; `from_vertices_faces()`, l. 221), `src/ray_tracing_2/main_triangles.py` (`_triangle_mesh_from_spec()`, l. 39; chamada a `TriangleMesh.from_vertices_faces()`, l. 68).

A síntese consolidada de aderência entre os conteúdos de geometria triangular e aceleração e a implementação atual foi reunida na Seção 9, junto aos quadros dos demais blocos conceituais.

5 Experimentos Breves

5.1 Cena mínima do núcleo geométrico

Arquivo âncora: `src/ray_tracing_2/main.py` (`render()`, l. 25; esfera, l. 54; plano, l. 55; luz pontual, l. 59).

Esta cena valida o núcleo mais simples do projeto: câmera pinhole, uma primitiva curva, um plano de apoio, iluminação local e sombra dura. Ela é a melhor cena para detectar regressões de geometria, orientação de normal e fluxo básico de `trace_ray()`.

5.2 Penumbra por fonte extensa

Arquivo âncora: `src/ray_tracing_2/main_area_light.py` (`render()`, l. 23; criação da `AreaLight`, l. 47–48).

Ao substituir a fonte pontual por uma fonte retangular amostrada, a borda de sombra deixa de ser descontínua e passa a ocupar uma região de penumbra. Esta

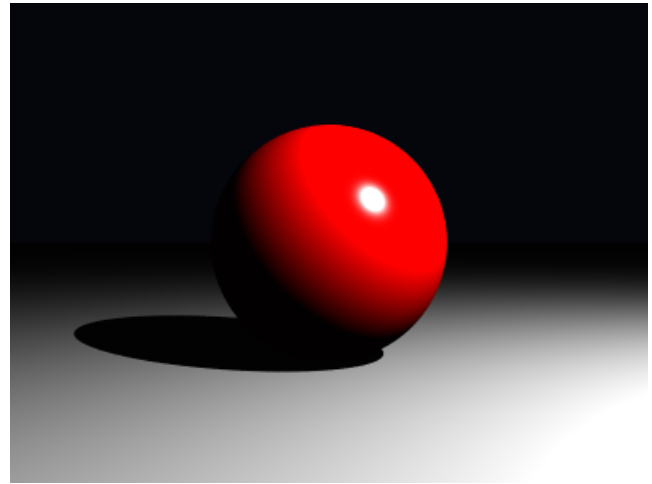


Figura 1. Cena mínima com `PointLight`.

imagem confirma empiricamente a leitura teórica sobre luz de área [Celes, 2026c, pp. 14–23]: a luz de área é uma integral sobre um emissor estendido aproximada por soma finita de amostras.

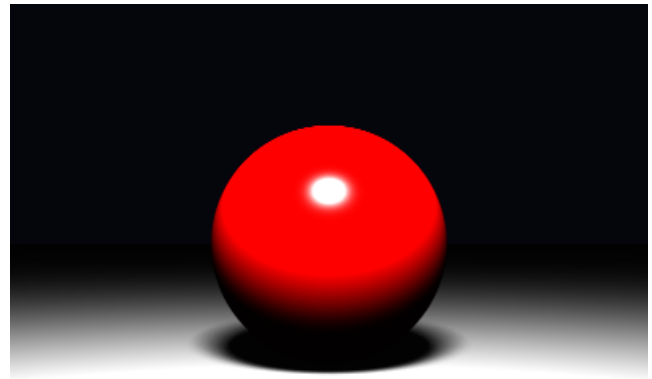


Figura 2. Penumbra com `AreaLight`.

5.3 Caixa tipo Cornell e instâncias geométricas

Arquivo âncora: `src/ray_tracing_2/main_box.py` (`render()`, l. 63; paredes, l. 116; blocos instanciados, l. 140; luz pontual, l. 143).

Esta cena é importante menos pelo efeito óptico avançado e mais pela infraestrutura geométrica: cinco paredes, dois blocos instanciados e uma configuração fechada que expõe erros de transformação, orientação de normal e auto-interseção. Ela demonstra que `Instance` permite reusar caixas canônicas como blocos orientados e transladados sem reescrever a matemática de interseção.

5.4 Malha triangular com BVH local

Arquivo âncora: `src/ray_tracing_2/main_triangles.py` (`_triangle_mesh_from_spec()`, l. 39; `TriangleMesh.from_vertices_faces()`, l. 68; `render()`, l. 100; luzes, l. 179).

Esta é a evidência experimental mais importante para a parte de triângulos e aceleração [Celes, 2026a,

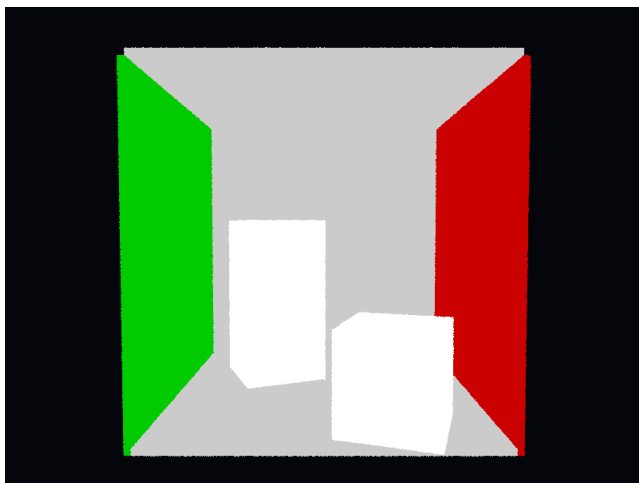


Figura 3. Cena tipo Cornell Box.

pp. 3–36]. O arquivo `outputs/main_triangles_20260424_233034/properties.md` registra 5 vértices, 6 faces trianguladas, acelerador `bvh`, `bvh_leaf_size = 2`, 7 nós, 4 folhas e profundidade máxima 3. Portanto, o artigo pode afirmar com segurança que a malha triangular e a BVH local estão de fato ativas na renderização dessa cena específica.

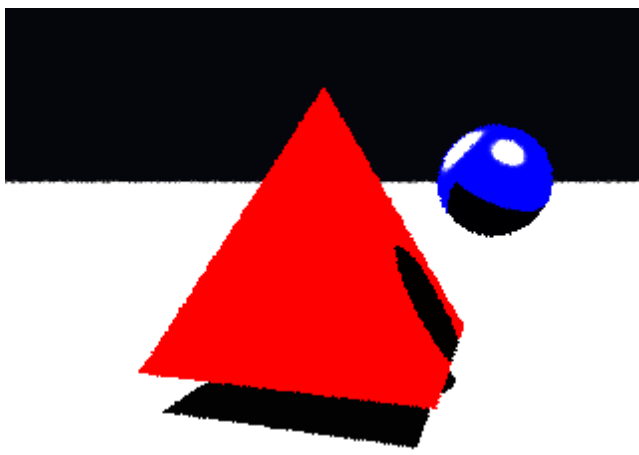


Figura 4. Malha triangular com BVH local.

5.5 Cornell Box com pirâmides refletiva e transparente

Arquivo âncora: `src/ray_tracing_2/cornell_box_pyramid.py` (`render()`, l. 50; construção das duas `TriangleMesh`, l. 167 e l. 174; luzes, l. 205 e l. 224).

Esta cena combina os dois eixos mais avançados do projeto: materiais recursivos [Celes, 2026c, pp. 26–35] e geometria triangulada [Celes, 2026a, pp. 3–14]. O arquivo `outputs/cornell_box_pyramid_20260425_003203/properties.md` confirma a presença de duas malhas triangulares distintas, uma transparente e outra refletiva, embutidas em uma cena fechada com múltiplas luzes. Isso a torna a melhor demonstração integrada de que triângulos, reflexão e refração não são módulos isolados, mas participam do mesmo pipeline de visibilidade e transporte local e recursivo.

Uma execução mais recente da mesma cena, agora com `--spp 10` e `--sampling_mode stratified`,

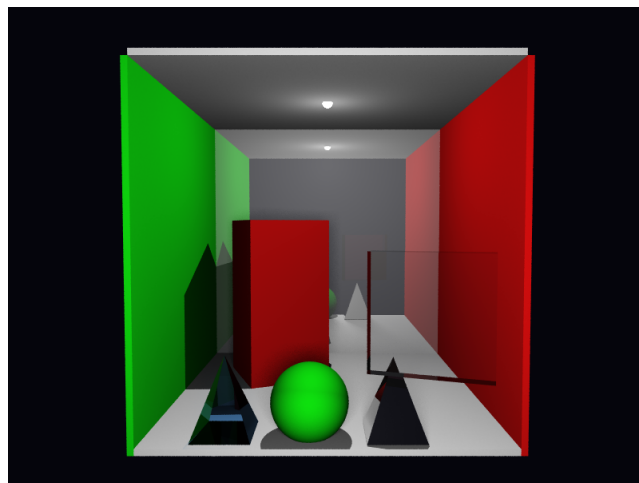


Figura 5. Cornell Box com duas pirâmides trianguladas.

reforça o mesmo ponto com uma evidência visual menos ruidosa e também documenta melhor o custo computacional da integração recursiva nessa configuração. O arquivo `outputs/cornell_box_pyramid_20260501_210143/properties.md` registra `samples_per_pixel = 10`, `sampling_mode = stratified`, `render_time_seconds = 1035.4509` e `render_time_minutes = 17.2575`, mantendo a mesma combinação central de duas `TriangleMesh`, uma transparente e outra refletiva, em uma Cornell Box com luz de área e luzes pontuais auxiliares.

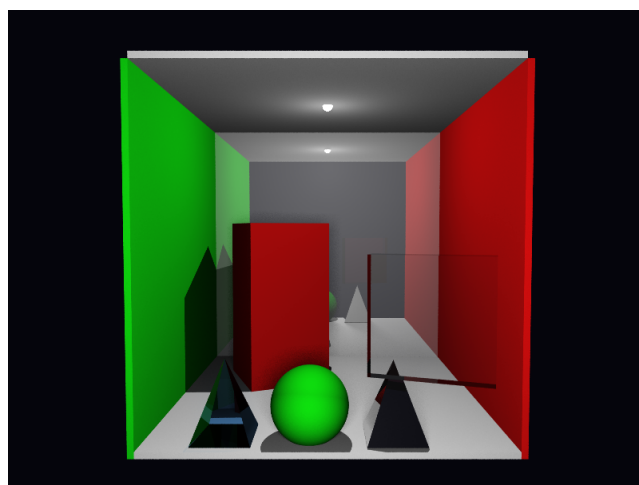


Figura 6. Cornell Box com duas pirâmides trianguladas em spp 10 estratificado.

5.6 Reprodutibilidade experimental por CLI padronizada

Embora a padronização do parser não altere a física do renderizador, ela melhora diretamente a rastreabilidade dos experimentos. Em particular, ela torna explícito um ponto conceitual importante do Slide 5: a amostragem bidimensional sobre o pixel [Celes, 2026c, pp. 4–9] e a amostragem bidimensional sobre a fonte extensa [Celes, 2026c, pp. 14–23] não são a mesma integral, ainda que ambas usem padrões 2D. Por isso, o repositório preserva dois controles públicos distintos, `sampling_mode` para o filme e `light_sampling_mode` para `AreaLight`, enquanto reutiliza apenas uma infraestrutura interna

compartilhada de geração de padrões 2D.

Do ponto de vista metodológico, essa separação é desejável: ela permite variar o estimador de anti-aliasing sem alterar a quadratura usada na integração da luz de área, e vice-versa. Isso reduz ambiguidade na interpretação dos resultados e melhora a comparabilidade entre execuções documentadas no próprio `properties.md`.

Exemplos mínimos de reprodução, todos com 800x600 e `spp = 1`:

```
python -m ray_tracing_2.main --width 800
--height 600 --spp 1
python -m ray_tracing_2.main_area_light
--width 800 --height 600 --spp 1
--sampling_mode stratified
--light_sampling_mode regular
python -m ray_tracing_2.cornell_box
--width 800 --height 600 --spp 1
--sampling_mode jittered
--light_sampling_mode uniform
python -m ray_tracing_2.main_triangles
--width 800 --height 600 --spp 1
--scene inputs/triangle_pyramid.json
--accelerator bvh
python -m ray_tracing_2.generate_scene
--input inputs/example_scene.json
--width 800 --height 600 --spp 1
```

Rastreabilidade no código. `src/ray_tracing_2/cli.py` (`build_parser()`, l. 14; `add_sampling_arguments()`, l. 37; `add_light_sampling_argument()`, l. 52), `src/ray_tracing_2/sampling.py` (`uniform_samples_2d()`, l. 15; `regular_grid_samples_2d()`, l. 26; `stratified_grid_samples_2d()`, l. 42), `src/ray_tracing_2/film.py` (`SamplingMode`, l. 17; `get_samples_for_pixel()`, l. 61), `src/ray_tracing_2/light.py` (`AreaLightSamplingMode`, l. 45; `iter_sample_uv()`, l. 114; `sample_radiance()`, l. 133), `src/ray_tracing_2/main.py` (`build_cli_parser()`, l. 79), `src/ray_tracing_2/main_area_light.py` (`build_cli_parser()`, l. 89), `src/ray_tracing_2/main_triangles.py` (`build_cli_parser()`, l. 202) e `src/ray_tracing_2/generate_scene.py` (`build_cli_parser()`, l. 122).

6 Limitações Atuais e Delimitação de Escopo

6.1 Estruturas de aceleração ausentes ou apenas parciais

À luz do Slide 6 [Celes, 2026a], a implementação atual cobre apenas uma fração do espaço de técnicas apresentado nos slides. O repositório não implementa grade regular, refinamento por SAT nem percorrimento incremental célula a célula [Celes, 2026a, pp. 15–21]; tampouco implementa SAH, compactação linear da BVH ou um agregador global de cena [Celes, 2026a, pp. 22–36]. O que existe de fato é uma BVH estática local a cada `TriangleMesh`, suficiente para reduzir o custo interno da malha, mas insuficiente para substituir uma estrutura de aceleração global do ray tracer.

Essa distinção é importante porque o custo no topo da cena continua linear. `Scene.compute_intersection()` percorre

`self.objects` de forma sequencial, de modo que a aceleração atua apenas dentro da malha triangulada, não entre esferas, caixas, instâncias e múltiplas malhas coexistindo na mesma cena. Portanto, a formulação correta no relatório não é “o Slide 6 foi implementado”, mas sim “parte do Slide 6 foi materializada por uma BVH local simplificada”.

Rastreabilidade no código. `src/ray_tracing_2/scene.py` (`compute_intersection()`, l. 24), `src/ray_tracing_2/triangle_bvh.py` (`TriangleBVH`, l. 140; `_build()`, l. 154; `TriangleBVHNode.intersect()`, l. 109).

6.2 Câmera e formação de imagem

O projeto permanece estritamente no modelo pinhole apresentado no material introdutório sobre câmera [Celes, 2026b, pp. 19–39]. Isso significa que `Camera.generate_ray()` sempre emite raios a partir de um único centro ótico, sem abertura finita, sem lente fina e sem profundidade de campo. A variável `focal_distance` controla apenas a escala geométrica do plano de projeção, e não um mecanismo de focalização seletiva.

Essa limitação não invalida o modelo adotado; ela apenas delimita seu alcance. O supersampling discutido no bloco de amostragem [Celes, 2026c, pp. 4–8] melhora a aproximação da média radiância por pixel, mas não altera a física óptica da câmera. Logo, qualquer leitura do resultado visual deve ser feita como imagem de câmera pinhole com amostragem estocástica, e não como imagem com desfoque óptico de lente.

Rastreabilidade no código. `src/ray_tracing_2/camera.py` (`Camera.__init__()`, l. 7; `generate_ray()`, l. 23), `src/ray_tracing_2/film.py` (`get_samples_for_pixel()`, l. 60).

6.3 Convenções radiométricas e simplificações de iluminação

Há uma simplificação deliberada e potencialmente enganosa se não for declarada: `PointLight` e `AreaLight` não seguem a mesma convenção radiométrica nesta base. Os slides de iluminação pontual apresentam o decaimento geométrico clássico por $1/r^2$ [Celes, 2026b, pp. 40–41], mas `PointLight.radiance()` preserva a convenção do enunciado principal e trata `power` como radiância incidente constante modulada apenas por visibilidade e transmitância. Já `AreaLight.sample_radiance()` distribui energia por amostras do emissor e introduz explicitamente o termo geométrico de distância.

O resultado prático é um subsistema de luzes internamente heterogêneo, embora coerente com os objetivos pedagógicos do projeto. Essa decisão é defensável para o Projeto 1, mas precisa ser descrita como simplificação de modelagem, e não como equivalência física entre fonte pontual ideal, emissor extenso e formulação radiométrica completa.

Rastreabilidade no código. `src/ray_tracing_2/light.py` (`PointLight.radiance()`, l. 46; `AreaLight.sample_radiance()`, l. 92).

6.4 Materiais recursivos e truncamento computacional

Os materiais reflexivo e transparente representam uma extensão substancial do núcleo local de Phong, mas permanecem aproximativos sob o ponto de vista da física de materiais. `ReflectiveMaterial.eval()` mistura Phong local e reflexão recursiva com Fresnel-Schlick de maneira plausível, porém não equivalente a um BRDF fisicamente baseado. `TransparentMaterial.eval()` usa Snell, TIR e Beer-Lambert, mas assume meio homogêneo, atenuação por canal RGB e ausência de dispersão espectral contínua.

Além disso, toda a recursão é truncada por `max_depth`. Isso é necessário para conter custo computacional e evitar crescimento exponencial do número de raios, mas significa que o traçador resolve apenas uma aproximação finita de cadeias longas de reflexão e transmissão. A leitura correta, portanto, é a de um ray tracer recursivo de profundidade limitada, não a de um solucionador completo de transporte de luz.

```
Rastreabilidade no código. src/ray_tracing_2/
material.py (ReflectiveMaterial.eval(), l.
101; TransparentMaterial.eval(), l. 186;
TransparentMaterial.shadow_transmittance(),
l. 167), src/ray_tracing_2/scene.py
(can_spawn_ray(), l. 41).
```

6.5 Leitura correta frente ao enunciado principal

Frente a `materiais/proj1.pdf`, a interpretação tecnicamente correta é que o repositório cobre com segurança o núcleo exigido pelo projeto e ainda incorpora várias extensões relevantes da disciplina: supersampling, instanciamento, luz de área, materiais recursivos, malha triangular e BVH local. Por outro lado, ele não esgota o espaço teórico apresentado nas aulas, sobretudo na parte de aceleração e na parte de modelagem física mais rigorosa da câmera, das fontes e dos materiais.

Essa delimitação é importante para o valor científico do artigo. O mérito do trabalho não está em inflar o escopo entregue, mas em mostrar com precisão onde a implementação coincide com a teoria ensinada, onde a simplifica e onde conscientemente para.

7 Diagramas como Camada Adicional de Rastreabilidade

Além das imagens de saída das cenas, o repositório inclui diagramas PlantUML efetivamente renderizados em PNG, que funcionam como documentação intermediária entre o texto deste artigo e o código executável. O ponto metodológico importante é que esses diagramas não substituem a análise anterior; eles a condensam em vistas complementares do pipeline real, já validadas sintaticamente pelo próprio PlantUML.

Visão geral do pipeline. Este diagrama sintetiza o fluxo entre `entrypoint`, `cli.py`, `CommonRenderOptions`, construção de cena e câmera, `Render.render()`, `Film.render()` e geração de `render.png`, `properties.json` e `properties.md`.

Fluxo dinâmico de sombreado recursivo. Esta vista separa do overview o trecho mais denso do material sobre reflexão, refração e transmitância, isto é, amostragem do filme, despacho por material, recursão por profundidade, reflexão, refração e transmitância em raios de sombra.

Fluxo local de aceleração para malhas. Este diagrama deixa explícito que a poda por AABB e a travessia da BVH acontecem dentro de `TriangleMesh`, e não como acelerador global da cena, o que reforça a leitura defendida na Seção 4.

Visão estrutural estática. Esta imagem complementa os diagramas dinâmicos ao mostrar dependências entre `Render`, `Scene`, `Film`, `Camera`, `Light`, `Material`, `TriangleMesh` e `TriangleBVH`.

Em conjunto, esses artefatos visuais fortalecem a rastreabilidade do artigo: as Seções 2 e 3 ganham uma representação operacional do pipeline completo, enquanto a Seção 4 passa a contar com uma visualização explícita da fronteira entre interseção global da cena e aceleração local de malhas triangulares.

8 Conclusão, Avaliação Final e Disponibilidade de Artefatos

O Projeto 1 implementa de forma sólida o núcleo geométrico e fotométrico do traçador e a maior parte das extensões ópticas e amostrais discutidas ao longo do artigo. O resultado é um ray tracer com câmera pinhole, interseções analíticas, Phong local, sombras duras, supersampling, instanciamento, luz de área, reflexão com Fresnel-Schlick, refração dielétrica com Snell e TIR, além de atenuação Beer-Lambert em raios refratados e de sombra.

O principal cuidado técnico desta versão é deixar claro o estatuto da geometria triangular e da aceleração frente às limitações consolidadas na Seção 6. O repositório entrega triangulação, `TriangleMesh` e uma BVH estática local construída por mediana no eixo dominante, com poda por AABB e travessia ordenada aproximadamente pela entrada do raio. Isso é suficiente para caracterizar uma aceleração real da malha, mas não autoriza afirmar implementação de grade regular, SAH, BVH linearizada ou agregador global de cena.

Tomando o estado atual do repositório como objeto de avaliação, a implementação entrega de forma efetiva um *ray tracer* educacional reproduzível, com núcleo geométrico-fotométrico completo, várias extensões relevantes do material da disciplina e uma camada operacional de execução suficientemente organizada para experimentação controlada. Em particular, o projeto já não deve ser lido apenas como um conjunto de cenas isoladas: ele possui um núcleo comum de CLI, uma convenção relativamente estável de geração de artefatos e uma fronteira explícita entre o que está fisicamente simplificado e o que está de fato operacionalizado no código. Os quadros consolidados apresentados a seguir organizam essa leitura em três tabelas: a Tabela 1 sintetiza os conteúdos do Slide 4, a Tabela 2 resume os conteúdos

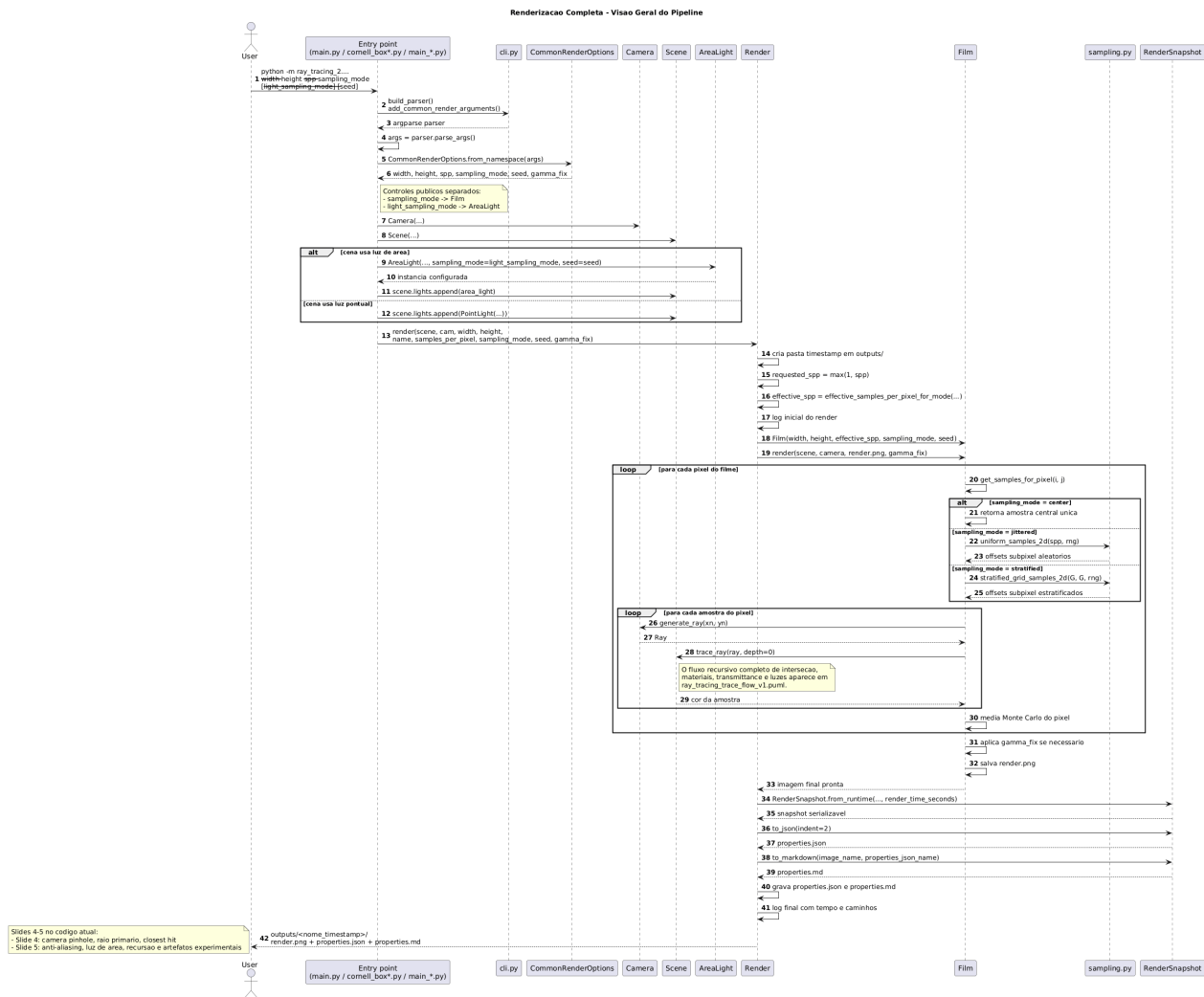
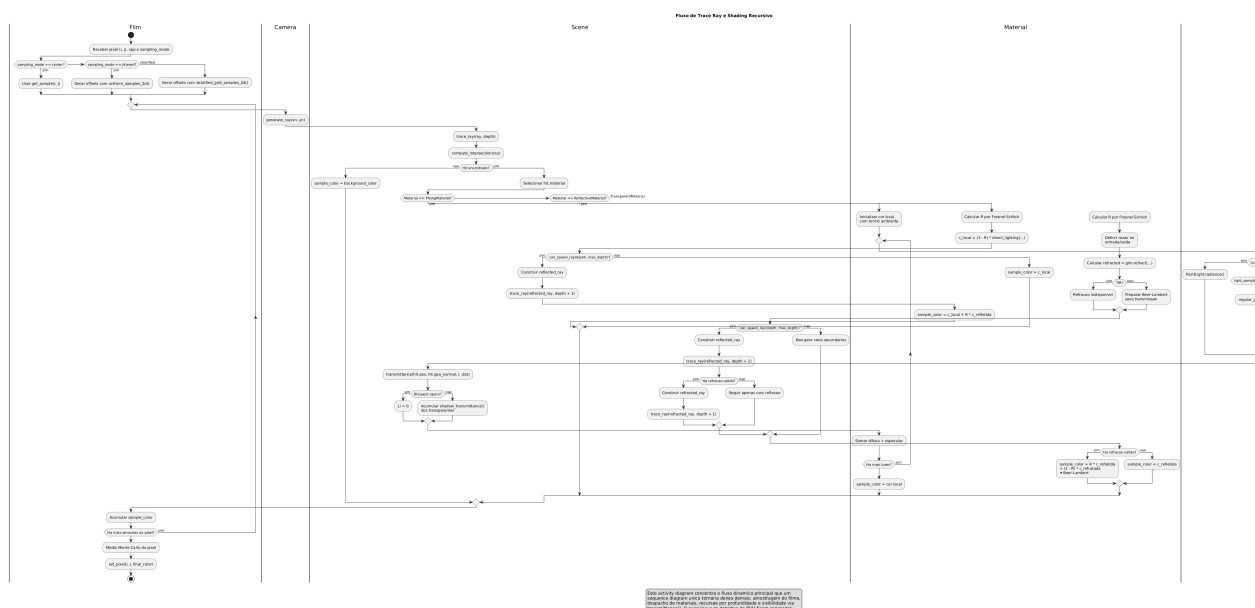


Figura 7. Diagrama renderizado da visão geral do pipeline.

Figura 8. Diagrama renderizado do fluxo de `trace_ray` e shading recursivo.

do Slide 5 e a Tabela 3 reúne o bloco de triângulos e estruturas de aceleração correspondente ao Slide 6.

Sob o critério do Projeto 1, essa entrega é técnica-

mente robusta e supera o núcleo mínimo ao incorporar supersampling, luz de área, materiais recursivos, geometria triangular e aceleração local por BVH. Sob o

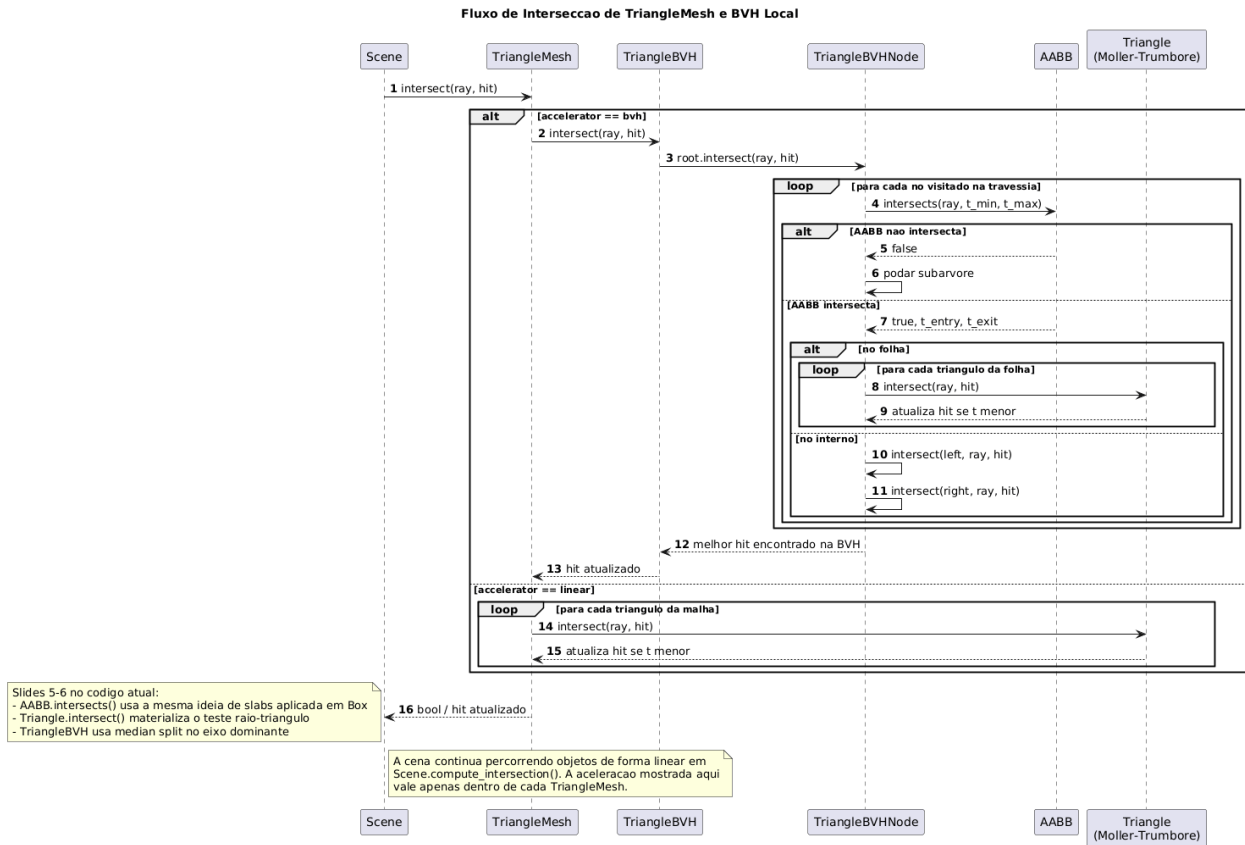


Figura 9. Diagrama renderizado do fluxo local de BVH para TriangleMesh.

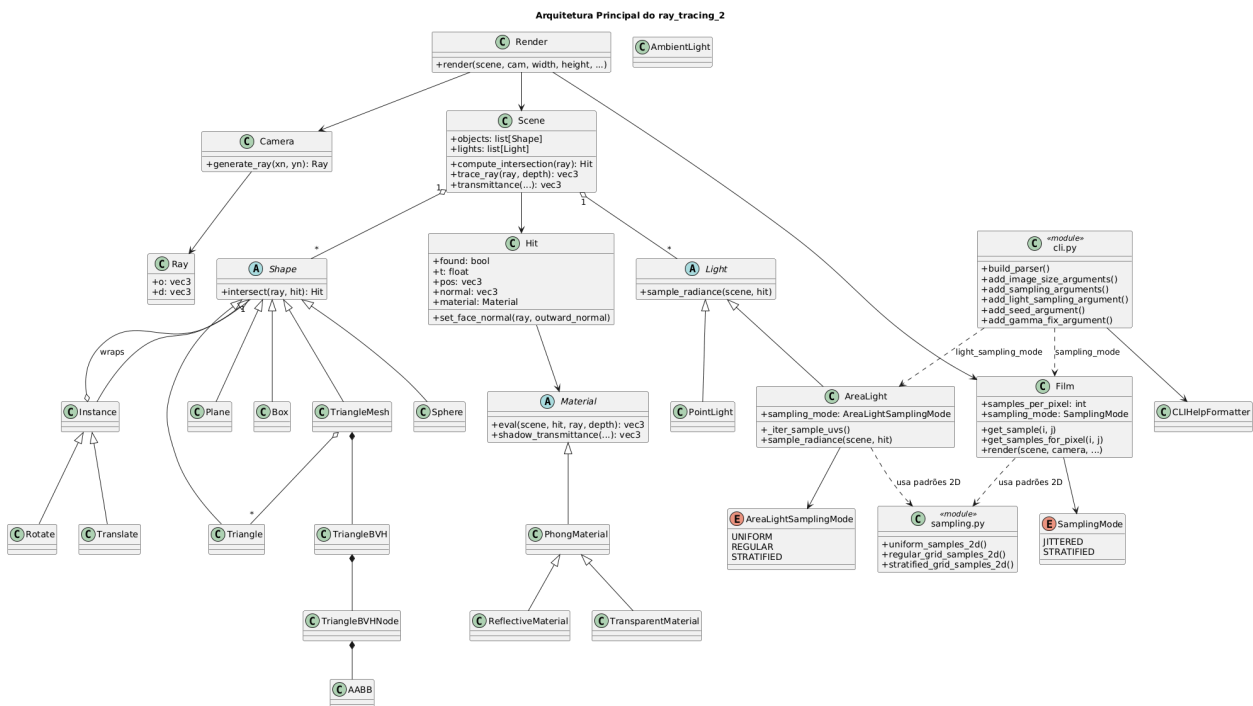


Figura 10. Diagrama renderizado da arquitetura principal do ray_tracing_2.

critério mais forte de cobertura integral do arco teórico das aulas, a entrega permanece conscientemente parcial. Essa parcialidade, contudo, não é caótica: ela é bem localizada. O que falta está concentrado em modelagem física mais rigorosa e em estruturas de aceleração mais sofisticadas, e não em inconsistência do pipeline princi-

pal.

Há também um ponto metodológico importante: a implementação agora entrega não apenas imagens, mas também rastreabilidade experimental. A combinação entre `src/ray_tracing_2/cli.py`, `src/ray_tracing_2/render.py` e `src/ray_tracing_2/render_`

`snapshot.py` faz com que cada execução relevante preserve resolução, modo de amostragem, semente, tempo de render e descrição estruturada da cena em artefatos persistentes. Isso aumenta o valor científico do projeto porque permite distinguir claramente entre efeito visual observado, parâmetro de entrada e hipótese geométrica ou física codificada.

Os artefatos de pesquisa discutidos neste artigo estão disponíveis no repositório do projeto [Miranda, 2026]. Em particular, o código-fonte analisado permanece versionado em `src/`, as cenas de entrada em `inputs/`, as renderizações e seus metadados persistidos em `outputs/` e os diagramas utilizados nesta discussão foram mantidos em formato renderizado junto ao repositório. Essa organização torna reproduzíveis tanto os resultados visuais quanto os parâmetros de execução usados na avaliação final.

Rastreabilidade	no	código.	<code>src/ray_tracing_2/cli.py</code>
			(<code>CommonRenderOptions;</code>
			<code>add_common_render_arguments()</code>),
			<code>src/ray_tracing_2/film.py</code>
			(<code>SamplingMode;</code>
			<code>get_samples_for_pixel()</code>),
			<code>src/ray_tracing_2/light.py</code>
			(<code>AreaLight;</code> <code>AreaLightSamplingMode;</code>
			<code>sample_radiance()</code>),
			<code>src/ray_tracing_2/render.py</code>
			(<code>Render.render()</code>),
			<code>src/ray_tracing_2/render_snapshot.py</code>
			(<code>RenderSnapshot.from_runtime()</code>);
			<code>to_markdown()</code>),
			<code>src/ray_tracing_2/shape.py</code>
			(<code>Triangle;</code> <code>TriangleMesh;</code> <code>Instance</code>),
			<code>src/ray_tracing_2/triangle_bvh.py</code>
			(<code>AABB;</code> <code>TriangleBVH</code>).

9 Quadros Consolidados de Aderência entre Referência e Implementação

Esta seção reúne, em formato sintético, os quadros de aderência deslocados das seções analíticas anteriores. Eles devem ser lidos como consolidação comparativa do que já foi discutido no corpo do artigo, em articulação com os slides da disciplina e com as referências selecionadas de *Physically Based Rendering* (PBRT 4e).

9.1 Núcleo geométrico e fotométrico

9.2 Extensões ópticas e amostrais

9.3 Geometria triangular e estruturas de aceleração

Declarações complementares

Contribuições dos autores

Yang Ricardo Barcellos Miranda foi responsável pela organização do manuscrito, consolidação da análise técnico-científica do repositório, revisão do alinhamento entre slides, PBRT e implementação, e preparação da versão atual em $\text{\LaTeX}$.

Uso de Inteligência Artificial

Ferramentas de IA generativa foram utilizadas como apoio à revisão textual, reorganização do conteúdo técnico e padronização da redação em $\text{\LaTeX}$. Todo o conteúdo técnico final foi revisado e validado pelo autor.

Referências

- Celes, W. (2026a). Estruturas de aceleração. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/6.estrutura_aceleracao.pdf. Acesso em: 1 maio 2026.
- Celes, W. (2026b). Traçado de raios. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/4.tracado_de_raios.pdf. Acesso em: 1 maio 2026.
- Celes, W. (2026c). Traçado de raios – parte ii. Slides de aula em PDF da disciplina Fundamentos de Computação Gráfica, hospedados no GitHub. Autor dos slides: W. Celes. Contato: celes@inf.puc-rio.br. Disponível em: https://github.com/yangricardo/INF2608-comp-grafica/blob/main/materiais/tra%C3%A7ado_de_raios/5.tracado_de_raios2.pdf. Acesso em: 1 maio 2026.
- Miranda, Y. R. B. (2026). Ray tracer educacional do projeto 1 de inf2608 – fundamentos de computação gráfica. Repositório GitHub do projeto. Disponível em: <https://github.com/yangricardo/INF2608-comp-grafica>. Acesso em: 1 maio 2026.
- Pharr, M., Jakob, W., and Humphreys, G. (2023). *Physically Based Rendering: From Theory to Implementation*. MIT Press, 4 edition. Online edition: <https://pbr-book.org/4ed/contents>.

Tabela 1. Quadro consolidado de aderência entre os conteúdos do Slide 4 e a implementação atual.

Conceito de referência	Situação no repositório	Evidência
Raio paramétrico e registro de hit	Implementado	<code>Ray</code> ; <code>Hit</code> ; <code>Scene.compute_intersection()</code>
Interseção raio-plano	Implementado	<code>Plane.intersect()</code>
Interseção raio-esfera	Implementado	<code>Sphere.intersect()</code>
Tolerância numérica para evitar auto-interseção	Implementado	<code>Scene.offset_point()</code> ; limiar positivo nas interseções
Câmera pinhole e geração de raios primários	Implementado	<code>Camera.__init__()</code> ; <code>Camera.generate_ray()</code>
Mudança de base entre espaço da câmera e espaço global	Implementado	<code>glm.lookAt()</code> + <code>inv_view</code> em <code>Camera</code>
Modelo local de Phong com ambiente, difusa e especular	Implementado	<code>PhongMaterial.direct_lighting()</code> ; <code>PhongMaterial.eval()</code>
Sombra dura por teste de visibilidade	Implementado com generalização	<code>Scene.transmittance()</code> reduz-se ao caso binário para opacos e estende o slide para transparentes
Luz pontual com síntese radiométrica do slide	Implementado com simplificação	<code>PointLight.radiance()</code> usa a convenção do enunciado sem queda explícita por $1/r^2$
Estrutura algorítmica básica câmera-hit-sombreamento	Implementado	<code>Film.render()</code> ; <code>Camera.generate_ray()</code> ; <code>Scene.trace_ray()</code>

Tabela 2. Quadro consolidado de aderência entre os conteúdos do Slide 5 e a implementação atual.

Conceito de referência	Situação no repositório	Evidência
Antialiasing por múltiplas amostras por pixel	Implementado com extensão	<code>sampling.uniform_samples_2d();</code> <code>sampling.stratified_grid_samples_2d();</code> <code>Film.get_samples_for_pixel();</code> <code>Film.render()</code>
Jitter subpixel aleatório	Implementado	<code>SamplingMode.JITTERED;</code> <code>sampling.uniform_samples_2d();</code> <code>Film.get_samples_for_pixel()</code>
Amostragem estratificada	Implementado como extensão	<code>SamplingMode.STRATIFIED;</code> <code>sampling.stratified_grid_samples_2d();</code> <code>Film.get_samples_for_pixel()</code>
Instanciação por transformações afins em coordenadas homogêneas	Implementado	<code>Instance.intersect();</code> <code>main_ellipse.py;</code> <code>main_box.py</code>
Transformação correta de normais por inversa transposta	Implementado	<code>m_inv_t</code> em <code>Instance.intersect()</code>
Luz de área com integração discreta e penumbra	Implementado com aproximação	<code>AreaLight.sample_radiance();</code> <code>AreaLight.radiance();</code> <code>main_area_light.py</code>
Padrões regular e uniforme puros para amostragem da luz	Implementado com modos explícitos	<code>AreaLightSamplingMode.REGULAR;</code> <code>AreaLightSamplingMode.UNIFORM;</code> <code>sampling.regular_grid_samples_2d();</code> <code>sampling.uniform_samples_2d();</code> <code>AreaLight._iter_sample_uvs()</code>
Caixa/AABB e método de slabs	Implementado	<code>Box.intersect();</code> <code>AABB.intersects()</code>
Traçador recursivo com profundidade finita	Implementado	<code>Scene.can_spawn_ray();</code> <code>Scene.trace_ray()</code>
Reflexão especular com Fresnel-Schlick	Implementado com aproximação	<code>ReflectiveMaterial.eval()</code>
Refração dielétrica com Snell, TIR e Beer-Lambert	Implementado com simplificação	<code>TransparentMaterial.eval();</code> <code>TransparentMaterial.shadow_transmittance()</code>
Generalização do shadow ray para transmitância acumulada	Implementado	<code>Scene.transmittance()</code>

Tabela 3. Quadro consolidado de aderência entre os conteúdos do Slide 6 e a implementação atual.

Conceito de referência	Situação no repositório	Evidência
Triângulo como primitiva elementar	Implementado	<code>Triangle</code> ; vetores de aresta $\mathbf{e}_1/\mathbf{e}_2$; normal geométrica
Interseção raio-triângulo por Möller-Trumbore	Implementado	<code>Triangle.intersect()</code>
Malha triangulada por vértices e faces	Implementado	<code>TriangleMesh</code> ; <code>TriangleMesh.from_vertices_faces()</code>
Integração da malha ao pipeline geral da cena	Implementado	<code>Scene.compute_intersection()</code> ; <code>main_triangles.py</code> ; <code>Instance.intersect()</code>
BVH local para malhas triangulares	Implementado com simplificação	<code>TriangleBVH</code> ; <code>TriangleBVHNode.intersect()</code> ; construção por mediana no eixo dominante
Caixas AABB como volumes de contenção	Implementado	<code>AABB</code> ; <code>AABB.intersects()</code>
Travessia recursiva com poda local por malha	Implementado com simplificação	ordem aproximada pela entrada do raio em <code>TriangleBVHNode.intersect()</code>
Grade regular para aceleração global	Não implementado	ausente no repositório; a cena continua sem grade uniforme global
SAH, BVH linearizada e agregador global de cena	Não implementado	<code>Scene.compute_intersection()</code> permanece sequencial; não há SAH nem estrutura global de aceleração